

Consumo de API con Retrofit Bottom Navigation, RecyclerView

Java · XML · Material Components · Retrofit 3 · PokéAPI

Requisitos

- Android Studio Quail 2 | 2026.1.2 Patch 1.
- Proyecto Empty Views Activity.
- Lenguaje Java y archivos de interfaz XML.
- SDK recomendado: Android 16, API 36.
- Emulador recomendado: Pixel 8, imagen Google Play x86_64.
- Conexión a internet.

PRIMERA PARTE. Crear el proyecto y configurar dependencias

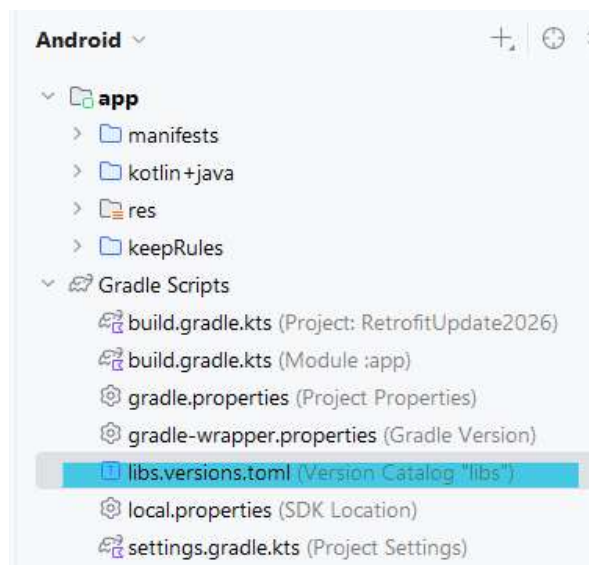
1. Crear el proyecto

Seleccione New Project > Phone and Tablet > Empty Views Activity y configure:

Campo	Valor sugerido
Name	PokeNavigation
Package name	co.edu.uniminuto.pokenavigation
Language	Java
Minimum SDK	API 24 o superior
Build configuration language	Kotlin DSL

2. Actualizar libs.versions.toml

Abra **gradle/libs.versions.toml**. Este proceso ya se hizo con volley, es muy parecido.

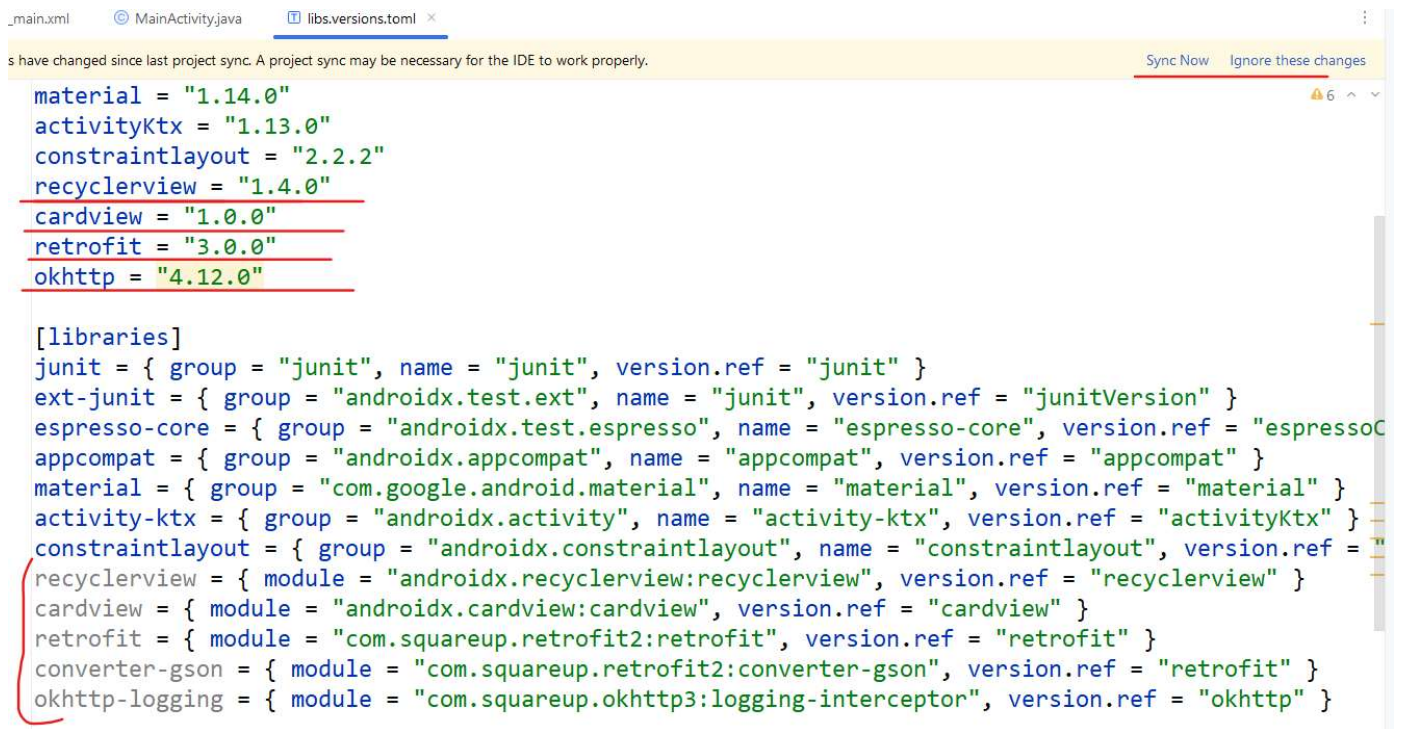


Conserve las versiones generadas por el proyecto y agregue:

```
[versions]
recyclerview = "1.4.0"
cardview = "1.0.0"
retrofit = "3.0.0"
okhttp = "4.12.0"

[libraries]
recyclerview = { module = "androidx.recyclerview:recyclerview", version.ref = "recyclerview" }
cardview = { module = "androidx.cardview:cardview", version.ref = "cardview" }
retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
converter-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "retrofit" }
okhttp-logging = { module = "com.squareup.okhttp3:logging-interceptor", version.ref = "okhttp" }
```

Imagen de referencia como debería quedar el archivo, luego de agregar las versiones y librerías hacer click en **Sync Now**:



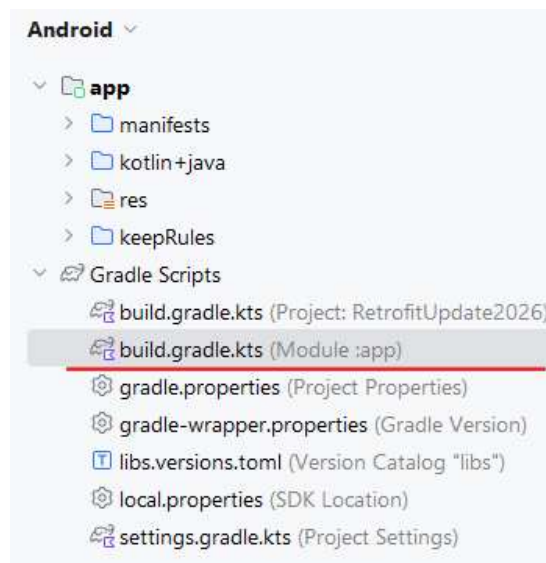
```
_main.xml  MainActivity.java  libs.versions.toml
s have changed since last project sync. A project sync may be necessary for the IDE to work properly.  Sync Now  Ignore these changes

material = "1.14.0"
activityKtx = "1.13.0"
constraintlayout = "2.2.2"
recyclerview = "1.4.0"
cardview = "1.0.0"
retrofit = "3.0.0"
okhttp = "4.12.0"

[libraries]
junit = { group = "junit", name = "junit", version.ref = "junit" }
ext-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }
material = { group = "com.google.android.material", name = "material", version.ref = "material" }
activity-ktx = { group = "androidx.activity", name = "activity-ktx", version.ref = "activityKtx" }
constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout", version.ref = "constraintlayout" }
recyclerview = { module = "androidx.recyclerview:recyclerview", version.ref = "recyclerview" }
cardview = { module = "androidx.cardview:cardview", version.ref = "cardview" }
retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
converter-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "retrofit" }
okhttp-logging = { module = "com.squareup.okhttp3:logging-interceptor", version.ref = "okhttp" }
```

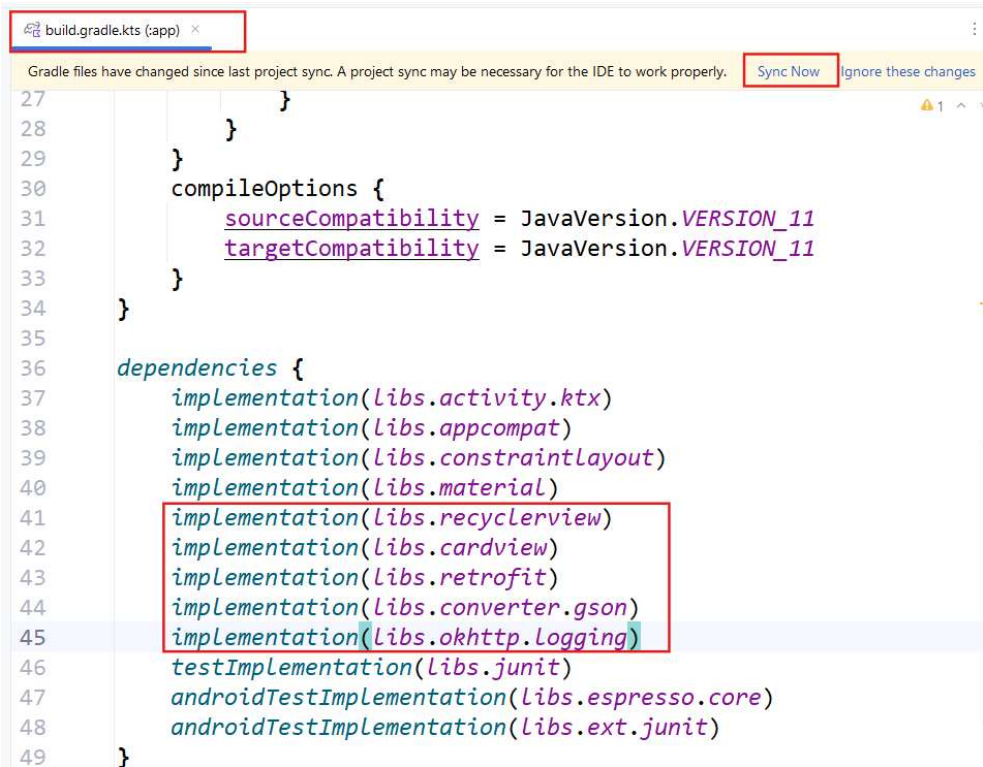
3. Agregar las dependencias del módulo app

Abra **app/build.gradle.kts** y agregue dentro de dependencies:



```
dependencies {
    implementation(libs.recyclerview)
    implementation(libs.cardview)
    implementation(libs.retrofit)
    implementation(libs.converter.gson)
    implementation(libs.okhttp.logging)
}
```

Pulse **Sync Now** y espere a que finalice la sincronización.



```
27     }
28 }
29 }
30 compileOptions {
31     sourceCompatibility = JavaVersion.VERSION_11
32     targetCompatibility = JavaVersion.VERSION_11
33 }
34 }
35
36 dependencies {
37     implementation(libs.activity.ktx)
38     implementation(libs.appcompat)
39     implementation(libs.constraintlayout)
40     implementation(libs.material)
41     implementation(libs.recyclerview)
42     implementation(libs.cardview)
43     implementation(libs.retrofit)
44     implementation(libs.converter.gson)
45     implementation(libs.okhttp.logging)
46     testImplementation(libs.junit)
47     androidTestImplementation(libs.espresso.core)
48     androidTestImplementation(libs.ext.junit)
49 }
```

4. Declarar el permiso de internet

En app/src/main/AndroidManifest.xml agregue el permiso antes de <application>:



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
4     <uses-permission android:name="android.permission.INTERNET" />
5     <application
```

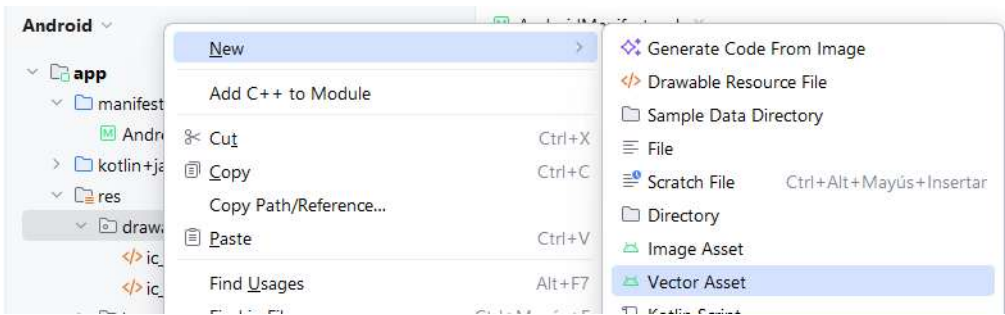
SEGUNDA PARTE. Bottom Navigation y Fragments

5. Crear los íconos

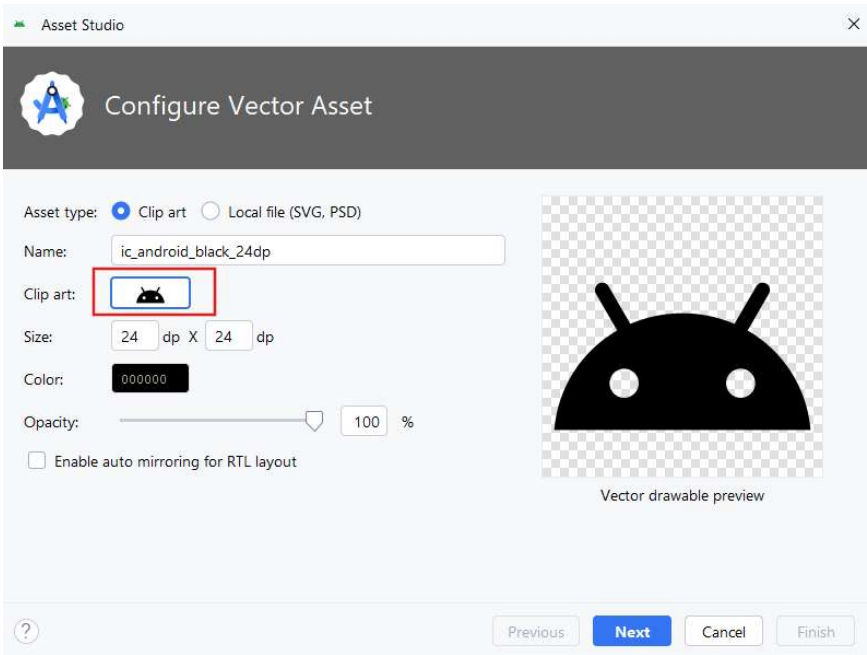
En res/drawable cree tres Vector Assets mediante New > Vector Asset:

Recurso	Ícono sugerido	Uso
ic_home.xml	home	Pantalla principal
ic_favorite.xml	favorite	Favoritos
ic_info.xml	info	Información

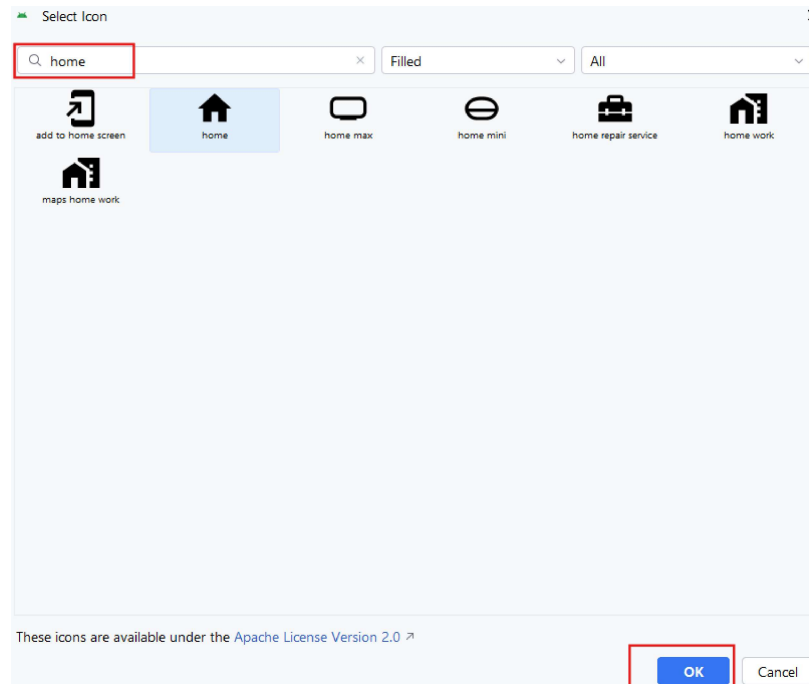
5.1 Para crear los vectores, botón derecho sobre el paquete drawable y seleccionar Vector Asset:



5.2 Hacemos click sobre el ícono de Clip art:



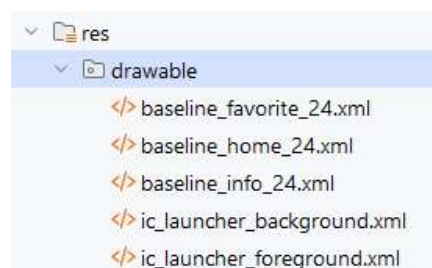
5.3 Buscar los íconos, hacemos click OK



5.4 Confirmar el Icono Path -> Finish



5.5 Ya tenemos los 3 vectores



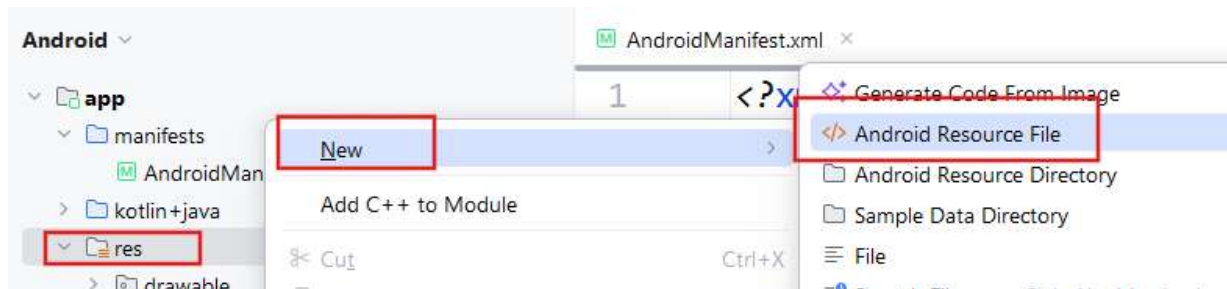
6. Crear los textos

En res/values/**strings.xml** agregue los texto para relacionar con los objetos de la vista:

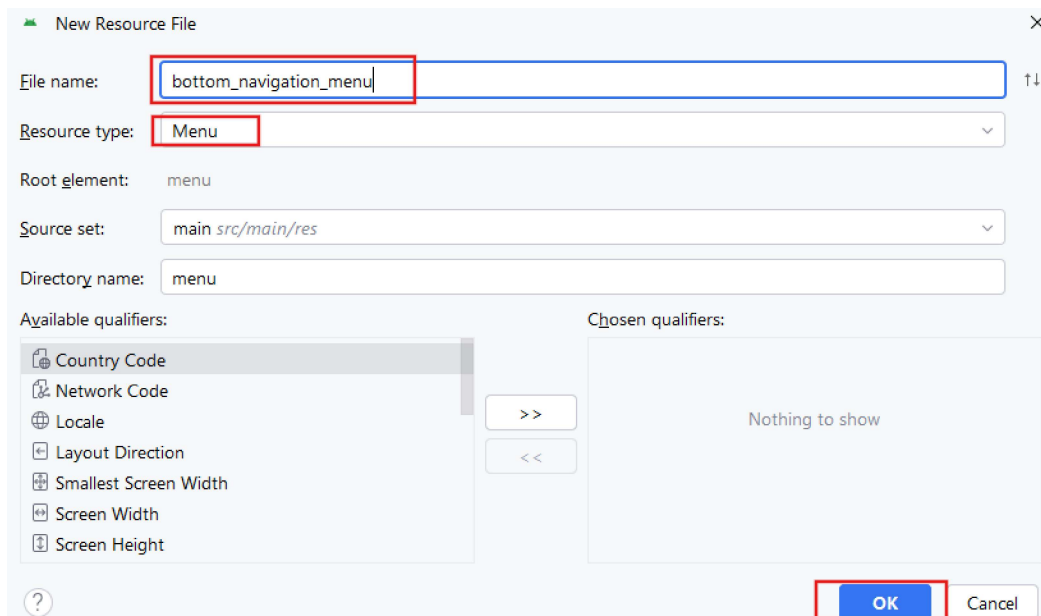
```
<resources>
    <string name="app_name">Poke Navigation</string>
    <string name="menu_inicio">Inicio</string>
    <string name="menu_favoritos">Favoritos</string>
    <string name="menu_informacion">Información</string>
    <string name="titulo_pokemon">Pokémon</string>
    <string name="cargando">Cargando información...</string>
    <string name="reintentar">Reintentar</string>
    <!-- TODO: Remove or change this placeholder text -->
    <string name="hello_blank_fragment">Hello blank fragment</string>
</resources>
```

7. Crear el menú de navegación

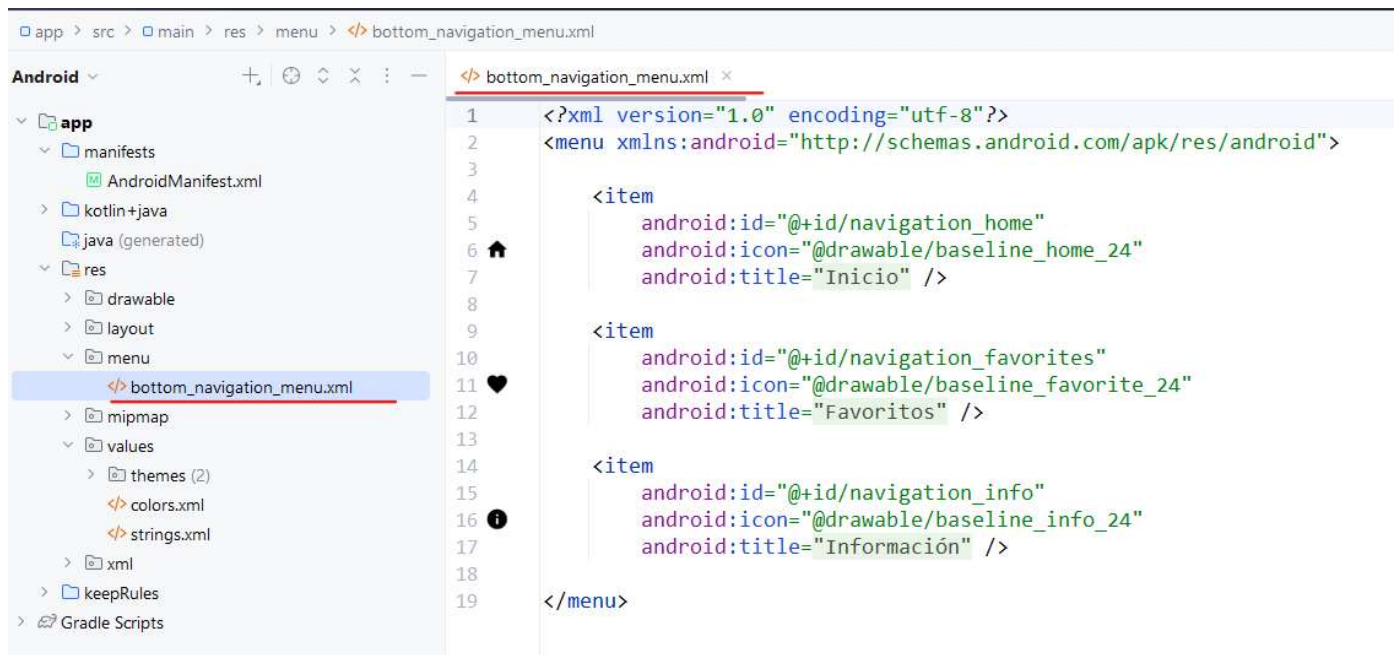
7.1 Vamos a crear el directorio res/menu y dentro agregue bottom_navigation_menu.xml:



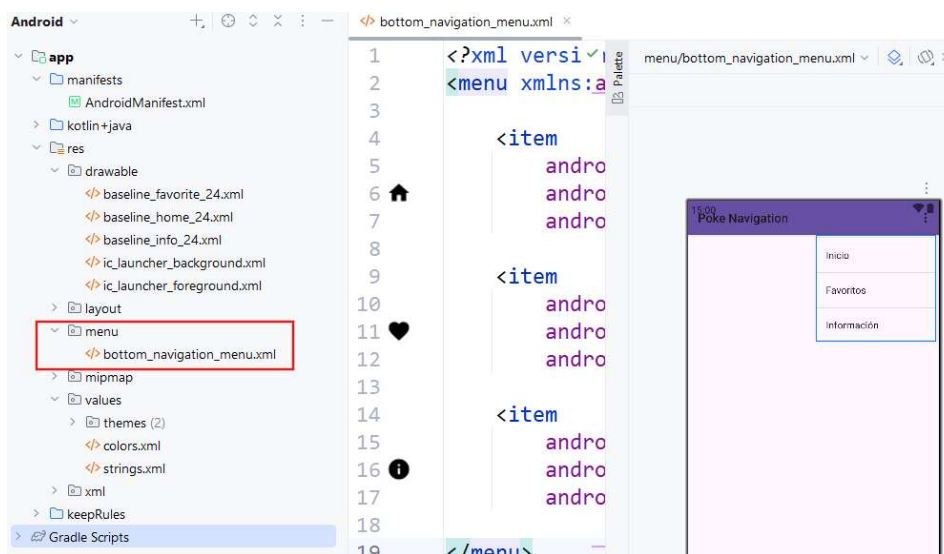
7.2 Colocamos el nombre del menú como se muestra en la imagen, tipo de recurso **Menu** y click **OK**



7.3 Vamos a la parte de código del **bottom_navigation_menu.xml** y colocamos el siguiente Código para crear el menu de navegación



Deberá verse algo parecido:

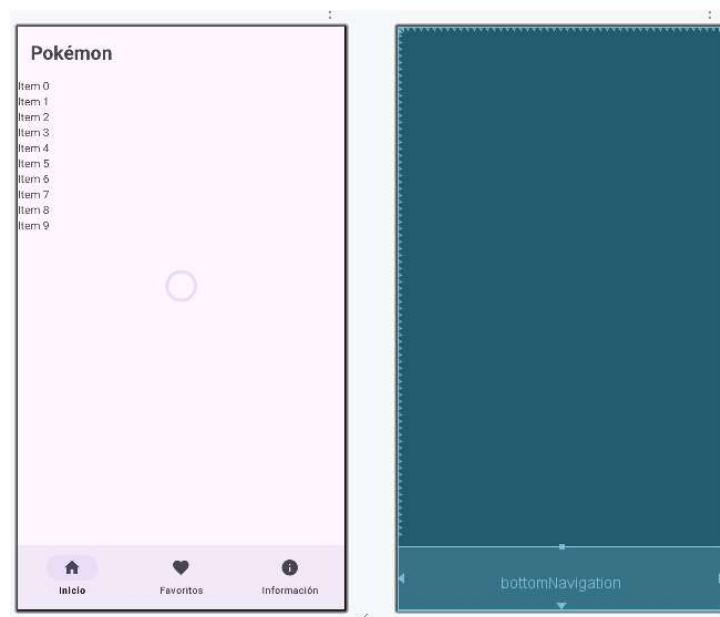


8. Diseñar activity_main.xml

La Activity contendrá un `FragmentManagerView` y un `BottomNavigationView`. Se utiliza `ConstraintLayout` para evitar medidas fijas y mejorar la adaptación.

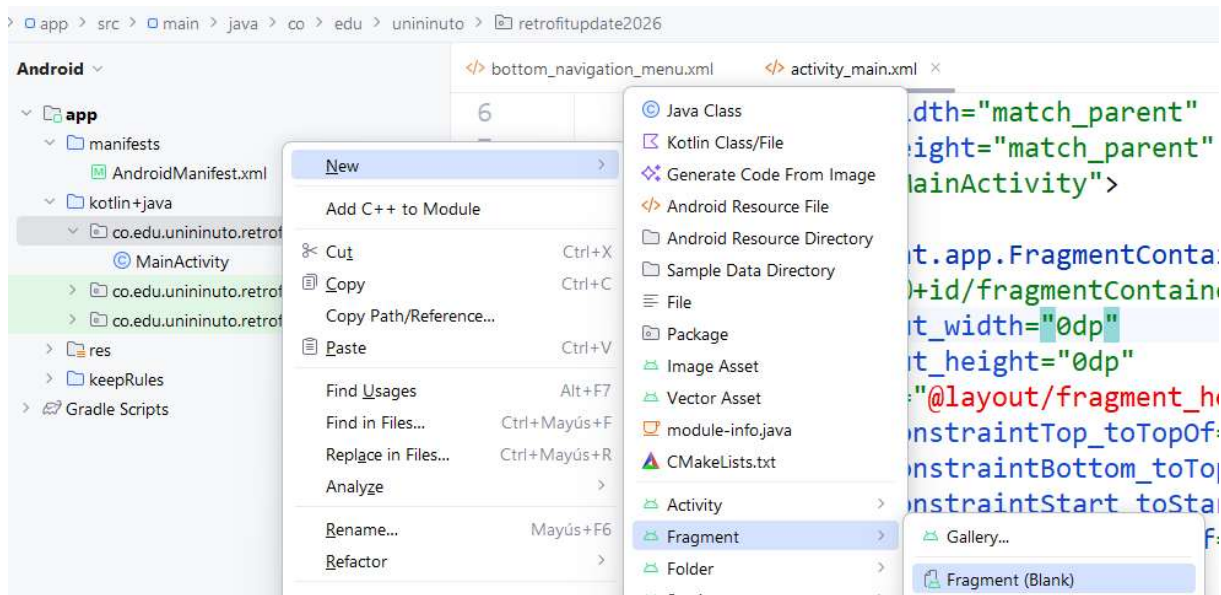
```
activity_main.xml x
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:id="@+id/main"
6      android:layout_width="match_parent"
7      android:layout_height="match_parent"
8      tools:context=".MainActivity">
9
10     <androidx.fragment.app.FragmentContainerView
11         android:id="@+id/fragmentContainer"
12         android:layout_width="0dp"
13         android:layout_height="0dp"
14         tools:layout="@layout/fragment_home"
15         app:layout_constraintTop_toTopOf="parent"
16         app:layout_constraintBottom_toBottomOf="@id/bottomNavigationView"
17         app:layout_constraintStart_toStartOf="parent"
18         app:layout_constraintEnd_toEndOf="parent" />
19
20     <com.google.android.material.bottomnavigation.BottomNavigationView
21         android:id="@+id/bottomNavigationView"
22         android:layout_width="0dp"
23         android:layout_height="wrap_content"
24         android:background="?attr/colorSurface"
25         app:labelVisibilityMode="labeled"
26         app:menu="@menu/bottom_navigation_menu"
27         app:layout_constraintBottom_toBottomOf="parent"
28         app:layout_constraintStart_toStartOf="parent"
29         app:layout_constraintEnd_toEndOf="parent" />
30
31 </androidx.constraintlayout.widget.ConstraintLayout>
```

Así debería de quedar:

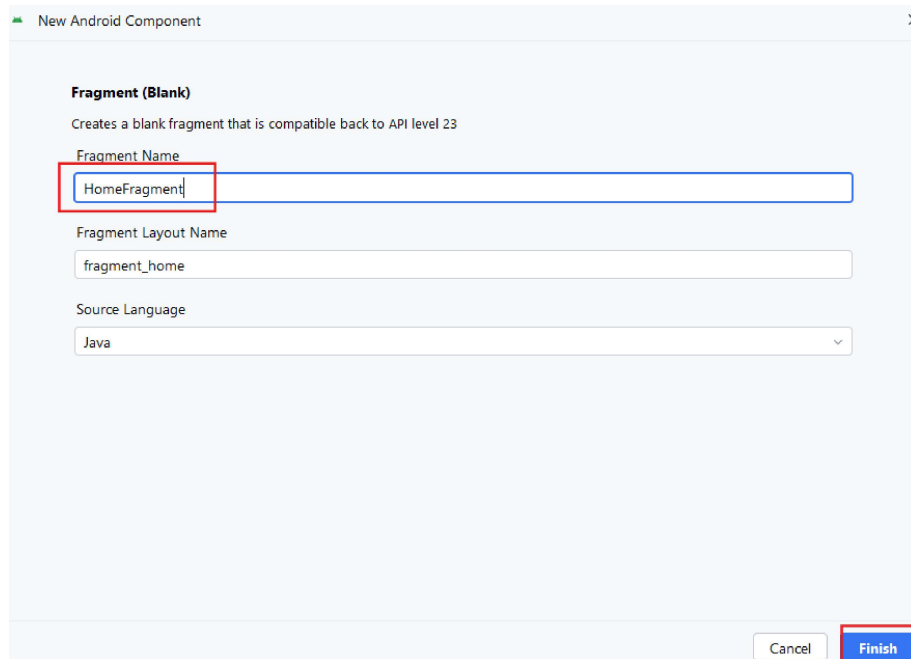


9. Crear los Fragments

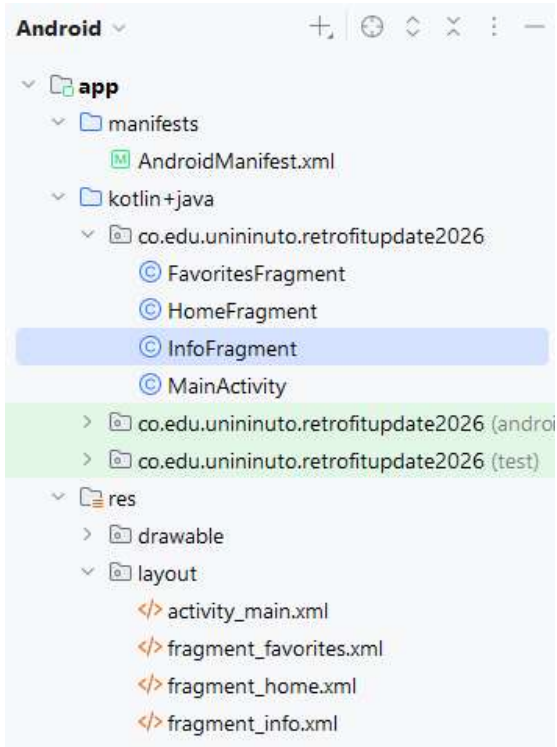
9.1 Cree tres clases: **HomeFragment**, **FavoritesFragment** e **InfoFragment**. Use New > Fragment > Fragment (Blank).



9.2 Colocar los nombres que se especificaron en el 9.1

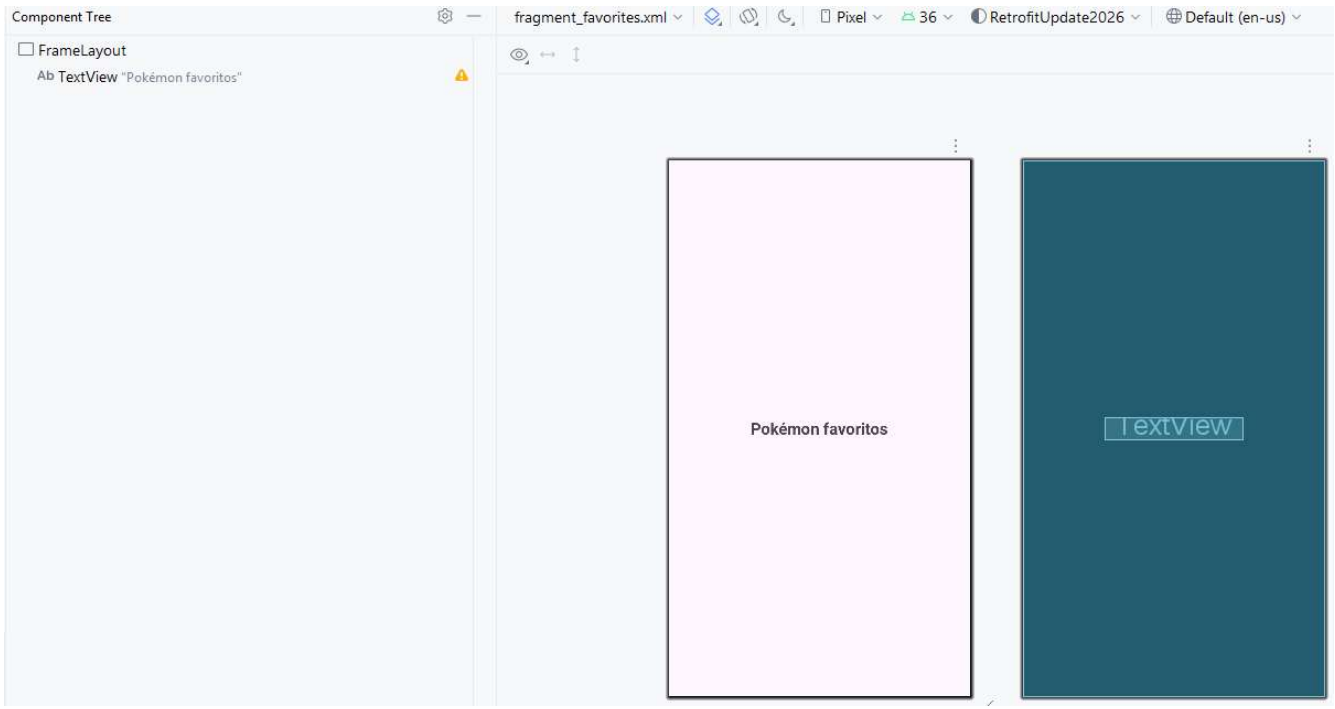


9.3 Este debería ser el resultado

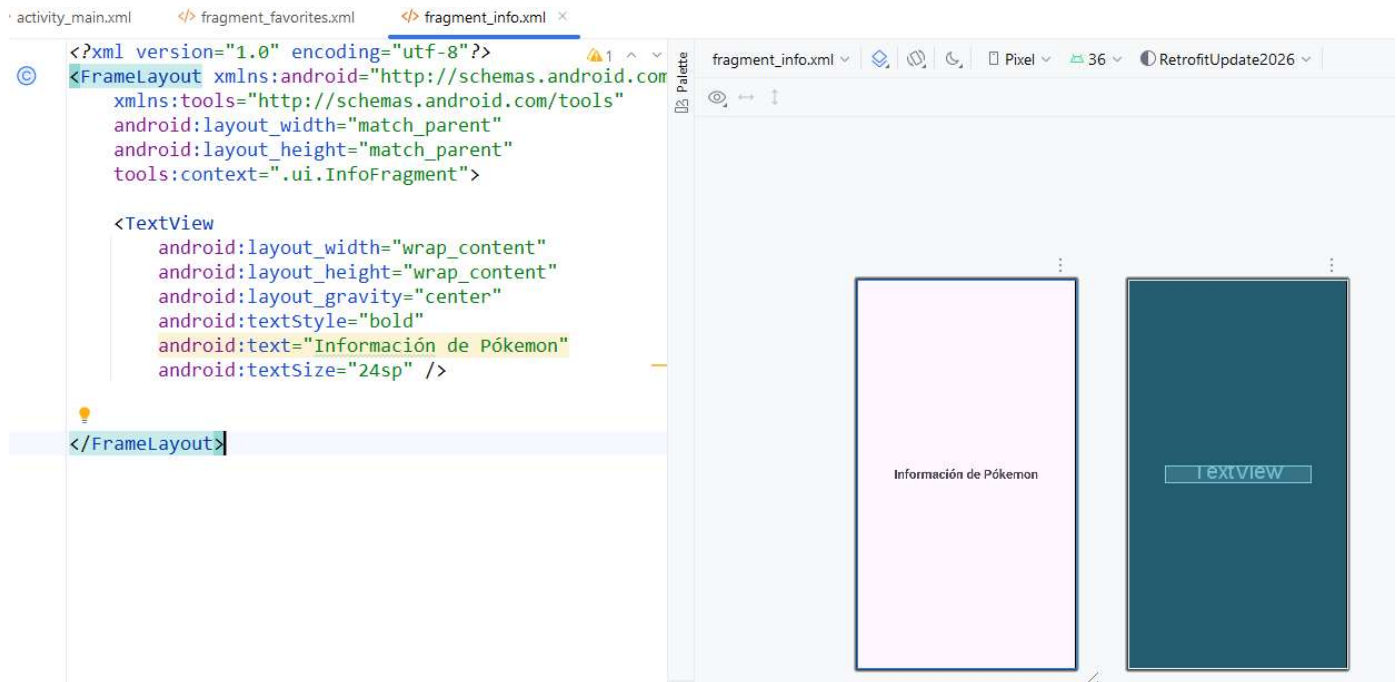


10. Diseños sencillos para Favoritos e Información

fragment_favorites.xml:



fragment_info.xml:



11. Programar MainActivity

```
package com.example.pokenavigation;

import android.os.Bundle;

import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;

import com.example.pokenavigation.ui.FavoritesFragment;
import com.example.pokenavigation.ui.HomeFragment;
import com.example.pokenavigation.ui.InfoFragment;
import com.google.android.material.bottomnavigation.BottomNavigationView;
```

11.1 Declaración de Atributos

```
public class MainActivity extends AppCompatActivity {

    2 usages
    private BottomNavigationView bottomNavigation;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

11.2 Crear el método de carga o enlaces a la Vista

```
1 usage
private void initObjects() {
    bottomNavigation = findViewById(R.id.bottomNavigation);
}
```

11.3 Método que recibe el id del item o fragmente el cual se ha seleccionado una opción en el menú

```
1 usage
private Fragment obtenerFragment(int itemId) {

    if (itemId == R.id.navigation_home) {
        return new HomeFragment();
    }

    if (itemId == R.id.navigation_favorites) {
        return new FavoritesFragment();
    }

    if (itemId == R.id.navigation_info) {
        return new InfoFragment();
    }

    return null;
}
```

11.4 Método de carga del Fragment

```
private void cargarFragment(Fragment fragment) {

    getSupportFragmentManager().beginTransaction()
        .replace(R.id.fragmentContainer, fragment)
        .commit();
}
```

11.5 Método de configuración de Bottom, aquí se debe saber cual fragment cargar

```
1 usage
private void configurarBottomNavigation() {

    bottomNavigation.setOnItemSelectedListener( MenuItem item -> {

        Fragment fragment = obtenerFragment(item.getItemId());

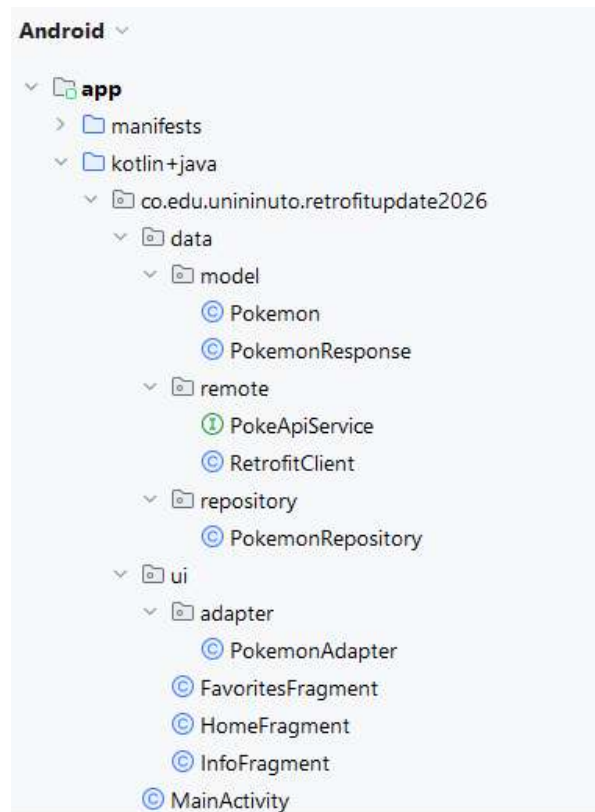
        if (fragment == null) {
            return false;
        }

        cargarFragment(fragment);

        return true;
    });
}
```

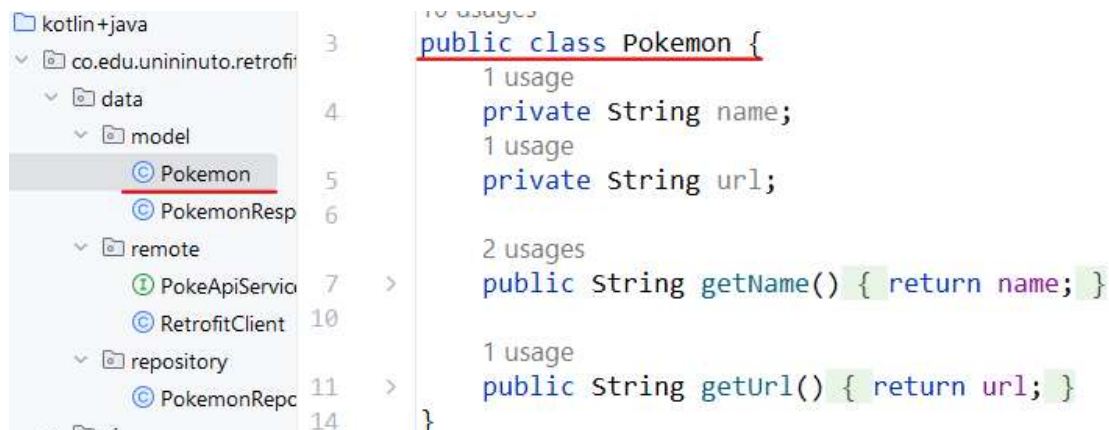
TERCERA PARTE. Modelo, CardView y RecyclerView

12. Crear la estructura de paquetes



13. Crear el modelo Pokemon

Dentro de data/model cree Pokemon.java:



14. Crear el contenedor de respuesta

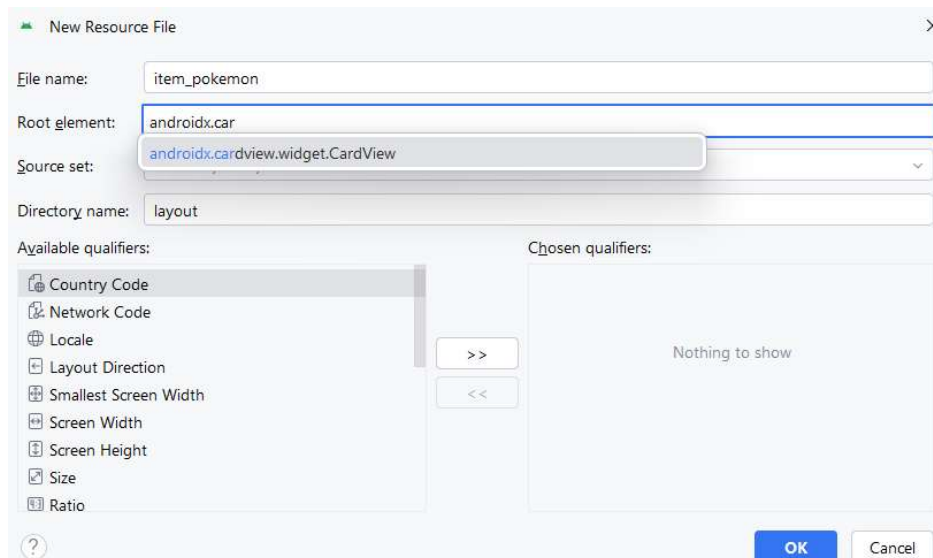
Cree PokemonResponse.java:

```
public class PokemonResponse {  
    1 usage  
    private int count;  
    no usages  
    private String next;  
    no usages  
    private String previous;  
    1 usage  
    private List<Pokemon> results;  
  
    no usages  
    public int getCount() {  
        return count;  
    }  
  
    no usages  
    public List<Pokemon> getResults() {  
        return results;  
    }  
}
```

Nota: La PokéAPI no retorna directamente una lista. Retorna un objeto que contiene count, next, previous y results. Por eso se necesita PokemonResponse.

15. Diseñar cada elemento del RecyclerView

15.1 Cree res/layout/item_pokemon.xml:



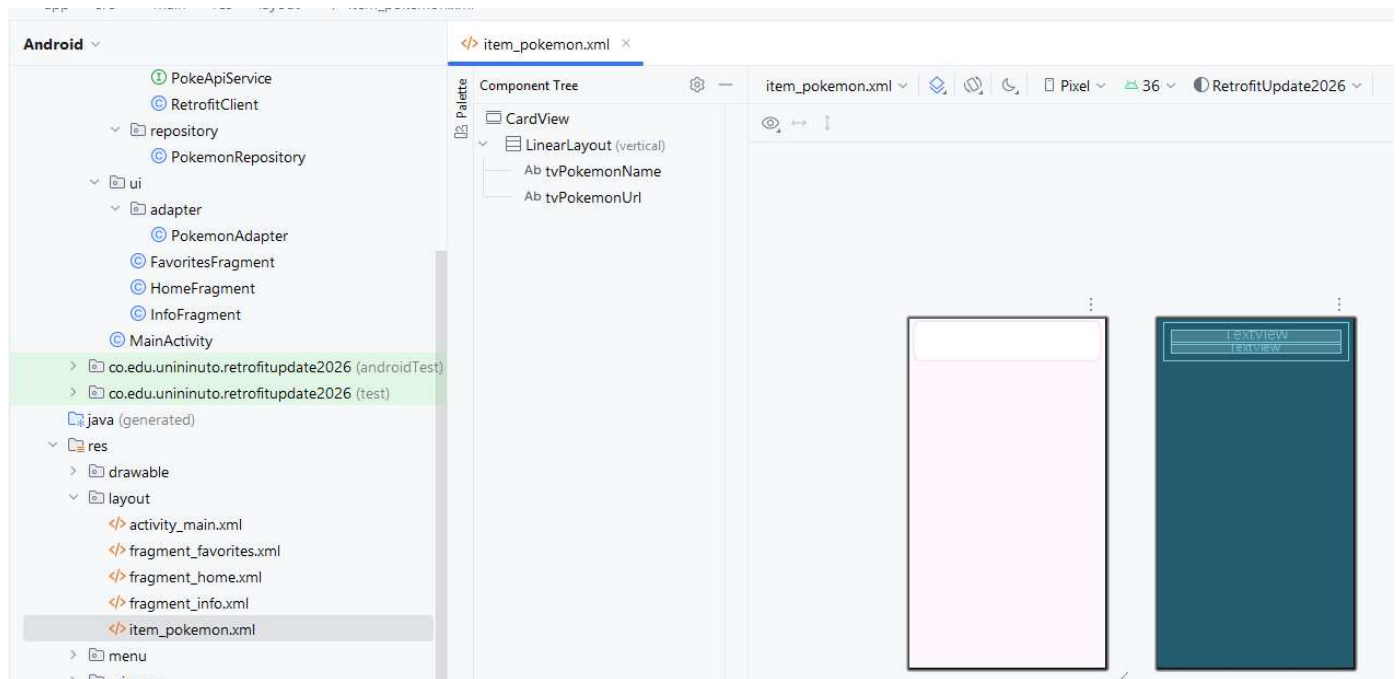
15.2 Ahora vamos al item_pokemon.xml

```
6 > app > src > main > res > layout > item_pokemon.xml

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_marginHorizontal="12dp"
    android:layout_marginVertical="6dp"
    app:cardCornerRadius="12dp"
    app:cardElevation="3dp"
    app:contentPadding="16dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">
        <TextView
            android:id="@+id/tvPokemonName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textSize="19sp"
            android:textStyle="bold" />
        <TextView
            android:id="@+id/tvPokemonUrl"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:ellipsize="end"
            android:maxLines="1"
            android:textSize="14sp" />
    </LinearLayout>
</androidx.cardview.widget.CardView>
```

Debería verse así:



16. Crear el adaptador

Dentro de ui/adapter cree PokemonAdapter.java:

```
package com.example.pokenavigation.ui.adapter;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.example.pokenavigation.R;
import com.example.pokenavigation.data.model.Pokemon;

import java.util.ArrayList;
import java.util.List;

public class PokemonAdapter
    extends RecyclerView.Adapter<PokemonAdapter.PokemonViewHolder> {

    public interface OnPokemonClickListener {
        void onPokemonClick(Pokemon pokemon);
    }

    private final List<Pokemon> pokemonList = new ArrayList<>();
    private final OnPokemonClickListener listener;

    public PokemonAdapter(OnPokemonClickListener listener) {
        this.listener = listener;
    }

    public void actualizarDatos(List<Pokemon> nuevosPokemon) {
        pokemonList.clear();

        if (nuevosPokemon != null) {
            pokemonList.addAll(nuevosPokemon);
        }

        notifyDataSetChanged();
    }

    @NonNull
    @Override
    public PokemonViewHolder onCreateViewHolder(
        @NonNull ViewGroup parent,
        int viewType
    ) {
        View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_pokemon, parent, false);

        return new PokemonViewHolder(view);
    }

    @Override
    public void onBindViewHolder(
        @NonNull PokemonViewHolder holder,
        int position
    ) {
        Pokemon pokemon = pokemonList.get(position);
        holder.bind(pokemon);
    }

    @Override
    public int getItemCount() {
        return pokemonList.size();
    }

    class PokemonViewHolder extends RecyclerView.ViewHolder {

        private final TextView tvPokemonName;
```

```

private final TextView tvPokemonUrl;

public PokemonViewHolder(@NonNull View itemView) {
    super(itemView);

    tvPokemonName = itemView.findViewById(R.id.tvPokemonName);
    tvPokemonUrl = itemView.findViewById(R.id.tvPokemonUrl);
}

public void bind(Pokemon pokemon) {
    tvPokemonName.setText(pokemon.getName());
    tvPokemonUrl.setText(pokemon.getUrl());

    itemView.setOnClickListener(
        view -> listener.onPokemonClick(pokemon)
    );
}
}
}

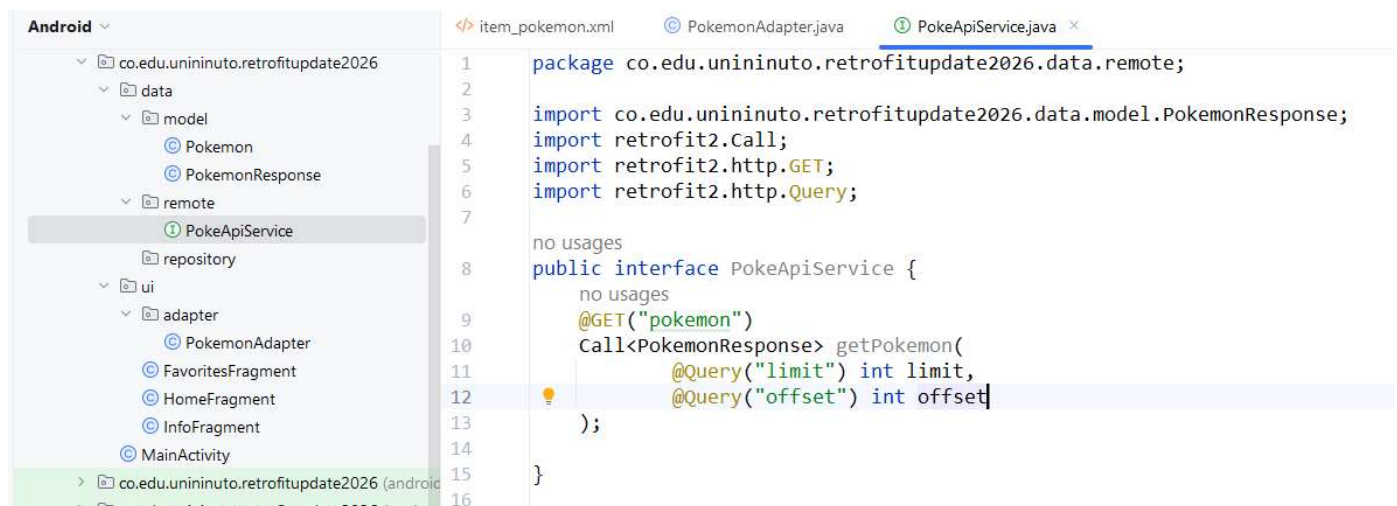
```

Nota: El clic se asocia al objeto Pokemon y no a una posición almacenada. Esto evita errores cuando la lista cambia o RecyclerView reutiliza sus vistas.

CUARTA PARTE. Configurar Retrofit

17. Crear la interfaz del servicio

Dentro de data/remote crear **PokeApiService.java**: en esta interface se definen las operaciones que va a consumir del API



18. Crear RetrofitClient

Dentro de el paquete remote crear la clase RetrofitCliente

```

import okhttp3.OkHttpClient;
import okhttp3.logging.HttpLoggingInterceptor;
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

```

```

public class RetrofitClient {
    // URL principal de la API
    1 usage
    private static final String BASE_URL = "https://pokeapi.co/api/v2/";
    // Instancia única de Retrofit
    3 usages
    private static Retrofit retrofit;
    // Constructor privado para evitar crear objetos de esta clase
    no usages
    private RetrofitClient() {
    }
    // Método para obtener el servicio de la API
    no usages
    public static PokeApiService getService() {
        // Retrofit se crea solamente una vez
        if (retrofit == null) {
            // Permite visualizar en Logcat las peticiones HTTP
            HttpLoggingInterceptor logging = new HttpLoggingInterceptor();
            // Muestra información básica:
            // método, URL y código de respuesta
            logging.setLevel(HttpLoggingInterceptor.Level.BASIC);
            // Cliente HTTP encargado de realizar las peticiones
            OkHttpClient client = new OkHttpClient.Builder()
                // Se agrega el interceptor de logs
                .addInterceptor(logging)
                .build();

            // Configuración de Retrofit
            retrofit = new Retrofit.Builder()
                // URL base de la API
                .baseUrl(BASE_URL)
                // Se utiliza el cliente OkHttpClient configurado
                .client(client)
                // Convierte el JSON de la API en objetos Java
                .addConverterFactory(GsonConverterFactory.create())
                // Construye Retrofit
                .build();
        }
        // Retrofit implementa automáticamente PokeApiService
        return retrofit.create(PokeApiService.class);
    }
}

```

Nota: La URL base debe terminar en /. El endpoint de @GET no debe comenzar con / porque debe concatenarse con la URL base.

19. Crear el repositorio

El repositorio separa la obtención de datos del Fragment. Dentro de data/repository crear PokemonRepository.java:

```
import co.edu.unininto.retrofitupdate2026.data.model.PokemonResponse;
import co.edu.unininto.retrofitupdate2026.data.remote.PokeApiService;
import co.edu.unininto.retrofitupdate2026.data.remote.RetrofitClient;
import retrofit2.Call;

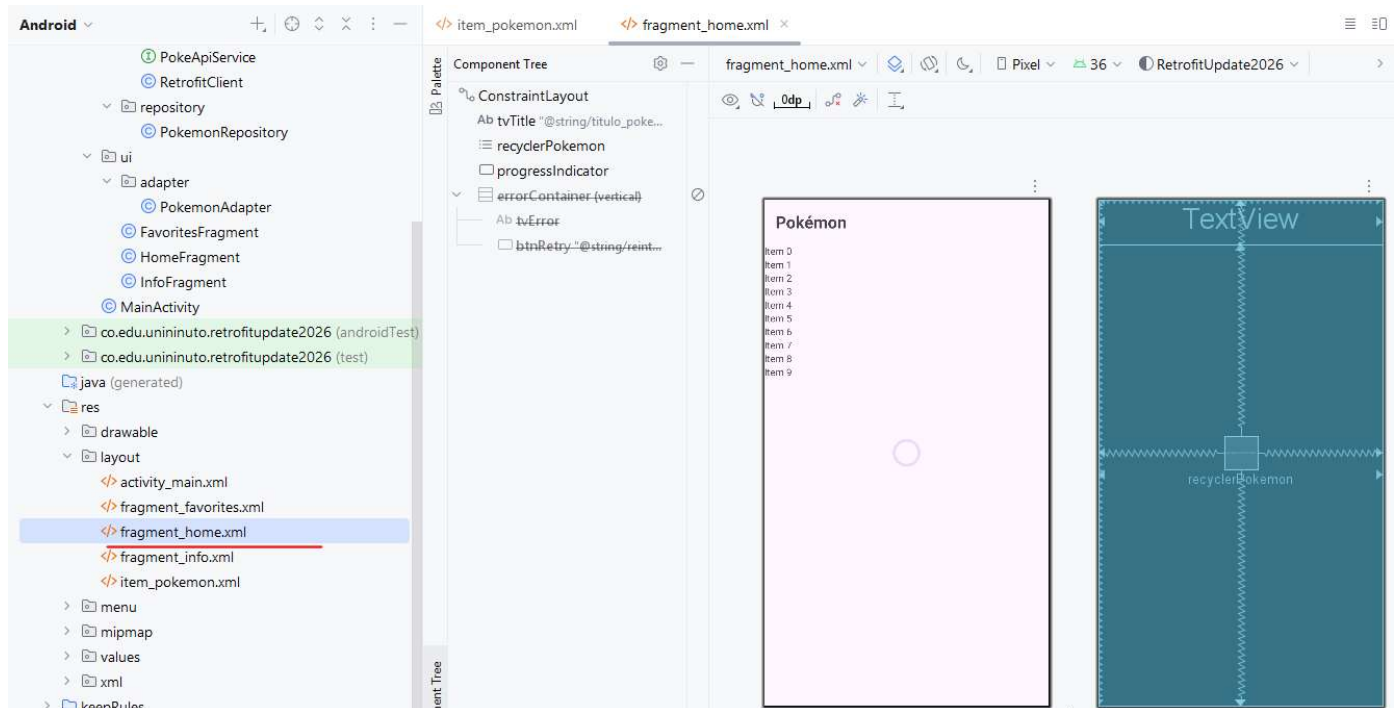
no usages
public class PokemonRepository {
    2 usages
    private final PokeApiService service;

    no usages
    public PokemonRepository() {
        service = RetrofitClient.getService();
    }

    no usages
    public Call<PokemonResponse> obtenerPokemon(
        int limit,
        int offset
    ) {
        return service.getPokemon(limit, offset);
    }
}
```

QUINTA PARTE. Pantalla principal y consumo de la API

20. Diseñar fragment_home.xml



Código xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/tools"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    tools:context=".ui.HomeFragment">
    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:padding="16dp"
        android:text="Pokémon"
        android:textSize="24sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerPokemon"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:clipToPadding="false"
        android:paddingBottom="12dp"
        app:layout_constraintTop_toBottomOf="@id/tvTitle"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <com.google.android.material.progressindicator.CircularProgressIndicator
        android:id="@+id/progressIndicator"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:contentDescription="Cargando información..."
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <LinearLayout
        android:id="@+id/errorContainer"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:gravity="center"
        android:padding="24dp"
        android:visibility="gone"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">

        <TextView
            android:id="@+id/tvError"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:gravity="center"
            android:textSize="17sp" />

        <com.google.android.material.button.MaterialButton
            android:id="@+id/btnRetry"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:text="Reintentar" />

    </LinearLayout>
</androidx.constraintlayout.widget.ConstraintLayout>
```

21. Programar HomeFragment

```
package com.example.pokenavigation.ui;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.pokenavigation.R;
import com.example.pokenavigation.data.model.Pokemon;
import com.example.pokenavigation.data.model.PokemonResponse;
import com.example.pokenavigation.data.repository.PokemonRepository;
import com.example.pokenavigation.ui.adapter.PokemonAdapter;
import com.google.android.material.button.MaterialButton;
import com.google.android.material.progressindicator.CircularProgressIndicator;

import retrofit2.Call;
import retrofit2.Callback;
import retrofit2.Response;

public class HomeFragment extends Fragment {

    private RecyclerView recyclerViewPokemon;
    private CircularProgressIndicator progressIndicator;
    private LinearLayout errorContainer;
    private TextView tvError;
    private MaterialButton btnRetry;

    private PokemonAdapter adapter;
    private PokemonRepository repository;
    private Call<PokemonResponse> currentCall;

    public HomeFragment() {
        super(R.layout.fragment_home);
    }

    @Override
    public void onViewCreated(
        @NonNull View view,
        @Nullable Bundle savedInstanceState
    ) {
        super.onViewCreated(view, savedInstanceState);

        recyclerViewPokemon = view.findViewById(R.id.recyclerPokemon);
        progressIndicator = view.findViewById(R.id.progressIndicator);
        errorContainer = view.findViewById(R.id.errorContainer);
        tvError = view.findViewById(R.id.tvError);
        btnRetry = view.findViewById(R.id.btnRetry);

        adapter = new PokemonAdapter(this::mostrarPokemonSeleccionado);
        repository = new PokemonRepository();

        recyclerViewPokemon.setLayoutManager(
            new LinearLayoutManager(requireContext())
        );
        recyclerViewPokemon.setHasFixedSize(true);
        recyclerViewPokemon.setAdapter(adapter);

        btnRetry.setOnClickListener(v -> cargarPokemon());

        cargarPokemon();
    }
}
```

```

private void cargarPokemon() {
    mostrarCargando();

    currentCall = repository.obtenerPokemon(30, 0);

    currentCall.enqueue(new Callback<>() {
        @Override
        public void onResponse(
            @NonNull Call<PokemonResponse> call,
            @NonNull Response<PokemonResponse> response
        ) {
            if (!isAdded()) {
                return;
            }

            PokemonResponse body = response.body();

            if (response.isSuccessful()
                && body != null
                && body.getResults() != null) {

                adapter.actualizarDatos(body.getResults());
                mostrarContenido();
            } else {
                mostrarError(
                    "No fue posible obtener los Pokémon. "
                    + "Código HTTP: " + response.code()
                );
            }
        }

        @Override
        public void onFailure(
            @NonNull Call<PokemonResponse> call,
            @NonNull Throwable throwable
        ) {
            if (call.isCanceled() || !isAdded()) {
                return;
            }

            mostrarError(
                "Error de conexión. Verifique internet "
                + "e intente nuevamente."
            );
        }
    });
}

private void mostrarPokemonSeleccionado(Pokemon pokemon) {
    Toast.makeText(
        requireContext(),
        "Seleccionó: " + pokemon.getName(),
        Toast.LENGTH_SHORT
    ).show();
}

private void mostrarCargando() {
    progressIndicator.setVisibility(View.VISIBLE);
    recyclerPokemon.setVisibility(View.GONE);
    errorContainer.setVisibility(View.GONE);
}

private void mostrarContenido() {
    progressIndicator.setVisibility(View.GONE);
    recyclerPokemon.setVisibility(View.VISIBLE);
    errorContainer.setVisibility(View.GONE);
}

private void mostrarError(String mensaje) {
    progressIndicator.setVisibility(View.GONE);
    recyclerPokemon.setVisibility(View.GONE);
    errorContainer.setVisibility(View.VISIBLE);
    tvError.setText(mensaje);
}

@Override

```

```

    public void onDestroyView() {
        if (currentCall != null) {
            currentCall.cancel();
        }

        recyclerPokemon.setAdapter(null);
        super.onDestroyView();
    }
}

```

Nota: La solicitud se cancela en onDestroyView para evitar que una respuesta intente modificar vistas que ya no existen.

22. Flujo de funcionamiento

Paso	Componente	Responsabilidad
1	HomeFragment	Solicita los datos al repositorio.
2	PokemonRepository	Invoca el método definido en PokeApiService.
3	RetrofitClient	Construye Retrofit, OkHttp y el convertidor Gson.
4	PokéAPI	Retorna un objeto JSON con la lista results.
5	PokemonAdapter	Muestra cada objeto en item_pokemon.xml.
6	Usuario	Selecciona un elemento y se ejecuta el listener.

23. Comprobación de la práctica

1. Ejecute la aplicación en el emulador Pixel 8.
2. Compruebe que la opción Inicio aparezca seleccionada.
3. Espere a que se muestre la lista de 30 Pokémon.
4. Seleccione un Pokémon y verifique el mensaje Toast.
5. Cambie entre Inicio, Favoritos e Información.
6. Desactive internet y pulse Reintentar para comprobar el estado de error.

24. Errores frecuentes

Problema	Causa probable	Solución
Expected BEGIN_ARRAY but was BEGIN_OBJECT	Se intentó recibir List<Pokemon> directamente.	Use PokemonResponse porque results está dentro de un objeto.
Base URL must end in /	La URL base no termina en diagonal.	Use https://pokeapi.co/api/v2/ .
Unable to resolve host	El dispositivo no tiene conexión o falta INTERNET.	Revise la red y AndroidManifest.xml.
RecyclerView no muestra datos	No tiene layoutManager o Adapter.	Configure ambos antes de cargar la API.
El menú no cambia de	Los id del menú no coinciden con	Revise navigation_home, navigation_favorites y

Fragment	MainActivity.	navigation_info.
La app falla después de cambiar de pantalla	La respuesta modifica vistas destruidas.	Cancele currentCall en onDestroyView.

Actividad propuesta

Amplíe la aplicación para que al seleccionar un Pokémon se abra un nuevo Fragment de detalle. Consuma el endpoint `pokemon/{name}` y muestre al menos la imagen oficial, altura, peso, experiencia base y tipos. Agregue una opción para guardar el Pokémon en la sección Favoritos.

Referencias

- Android Developers. Android Studio Quail 2 y compatibilidad con Android Gradle Plugin.
- Android Developers. RecyclerView.
- Square. Retrofit.
- PokéAPI. RESTful Pokémon API.
- Material Design. Bottom Navigation y Progress Indicators.